

Aprofundamento – Deep Learning

Rafael Stutz

Aula 3 · 07/10/2026

Deep Learning (DEPAI) · IASEG

1. Abertura: o que vocês já precisam saber e onde estão as dúvidas;
2. Funções de ativação: o que são e quando usar cada uma;
3. Funções de perda: o que são e quando usar cada uma;
4. Taxa de aprendizado: o tamanho do passo;
5. Lote e época;
6. Juntando tudo: estudos de caso e o código no Keras;
7. Kahoot e o que levar desta aula.

Abertura

O que vocês já precisam saber

Da Aula 1

- **Neurônio:** soma ponderada mais bias, seguida de uma função de ativação;
- **Rede:** camadas em sequência;
- Sem função de ativação, a rede é uma **reta**;
- **Treinar:** ajustar os pesos para reduzir a perda.

Da Aula 2

- **Parâmetros** são aprendidos; **hiperparâmetros** são escolhidos antes;
- **Treino, validação e teste.** Sobreajuste: bom no treino, ruim na validação;
- **Classificação** (uma classe) ou **regressão** (um número);
- **Keras:** *Sequential, Dense, compile, fit.*

Alguma dessas frases não está clara? É a hora de perguntar.

Para começar

Para discutir

1. Em uma frase: o que acontece quando chamamos *modelo.fit*?
2. Qual destas palavras vocês teriam mais dificuldade de explicar a um colega?

ativação

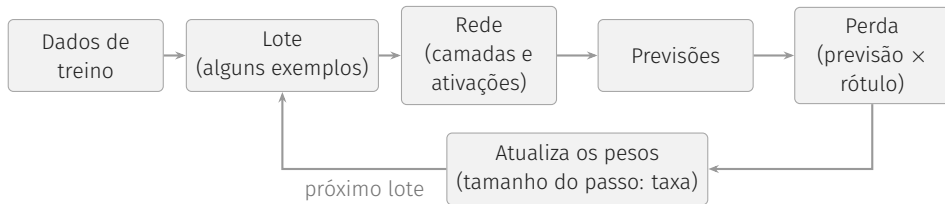
perda

taxa de
aprendizado

lote

época

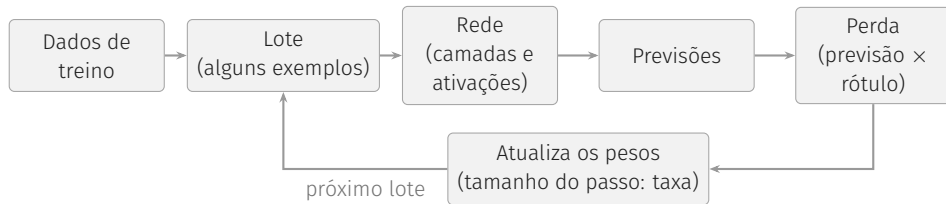
O treino em uma figura



todos os lotes uma vez = **1 época**; o treino repete por várias épocas

Exemplo: 1.000 ocorrências rotuladas, lotes de 100.

O treino em uma figura

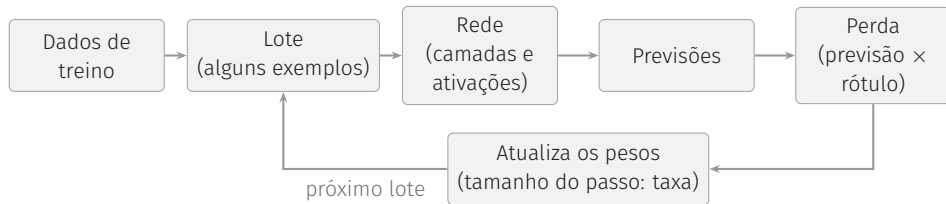


todos os lotes uma vez = **1 época**; o treino repete por várias épocas

Exemplo: 1.000 ocorrências rotuladas, lotes de 100.

1. Pega 100 ocorrências: um **lote**;

O treino em uma figura

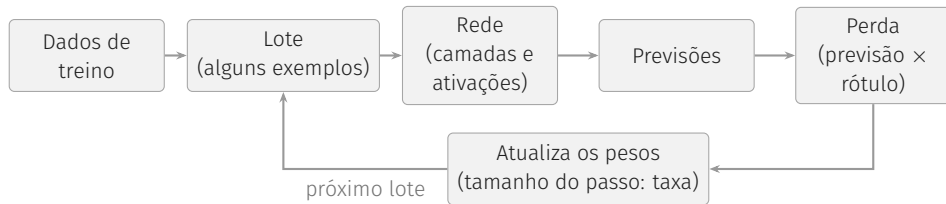


todos os lotes uma vez = **1 época**; o treino repete por várias épocas

Exemplo: 1.000 ocorrências rotuladas, lotes de 100.

1. Pega 100 ocorrências: um **lote**;
2. A rede calcula uma previsão, camada por camada: soma ponderada e **ativação**;

O treino em uma figura

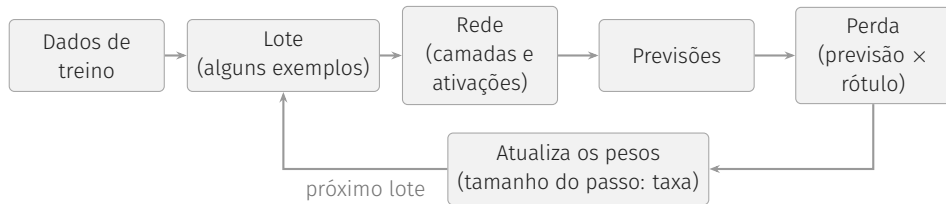


todos os lotes uma vez = **1 época**; o treino repete por várias épocas

Exemplo: 1.000 ocorrências rotuladas, lotes de 100.

1. Pega 100 ocorrências: um **lote**;
2. A rede calcula uma previsão, camada por camada: soma ponderada e **ativação**;
3. A **perda** resume o erro das 100 previsões em **um número**;

O treino em uma figura

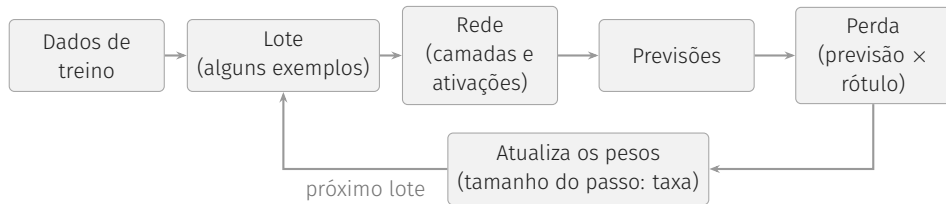


todos os lotes uma vez = **1 época**; o treino repete por várias épocas

Exemplo: 1.000 ocorrências rotuladas, lotes de 100.

1. Pega 100 ocorrências: um **lote**;
2. A rede calcula uma previsão, camada por camada: soma ponderada e **ativação**;
3. A **perda** resume o erro das 100 previsões em **um número**;
4. Os pesos mudam um pouco, para reduzir a perda: a **taxa** diz quanto;

O treino em uma figura

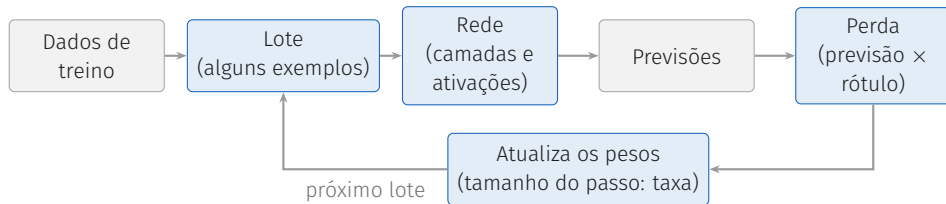


todos os lotes uma vez = **1 época**; o treino repete por várias épocas

Exemplo: 1.000 ocorrências rotuladas, lotes de 100.

1. Pega 100 ocorrências: um **lote**;
2. A rede calcula uma previsão, camada por camada: soma ponderada e **ativação**;
3. A **perda** resume o erro das 100 previsões em **um número**;
4. Os pesos mudam um pouco, para reduzir a perda: a **taxa** diz quanto;
5. Próximo lote. Depois de 10 lotes, a rede viu tudo: **1 época**.

Os botões de hoje

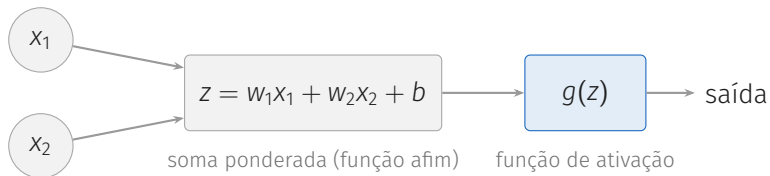


todos os lotes uma vez = **1 época**; o treino repete por várias épocas

Botão	No Keras	Pergunta que ele responde
Função de ativação	<code>Dense(..., activation=)</code>	que formas a rede desenha? o que a saída significa?
Função de perda	<code>compile(loss=)</code>	o que conta como erro, e quanto?
Taxa de aprendizado	<code>Adam(learning_rate=)</code>	de que tamanho é cada passo?
Lote e época	<code>fit(batch_size=, epochs=)</code>	quantos exemplos por passo? quantas voltas?

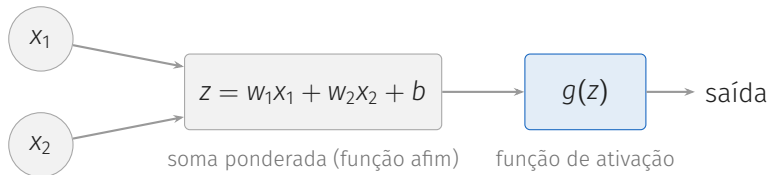
Funções de ativação

Onde fica a função de ativação



A função de ativação tem **dois trabalhos**, conforme o lugar da camada:

Onde fica a função de ativação

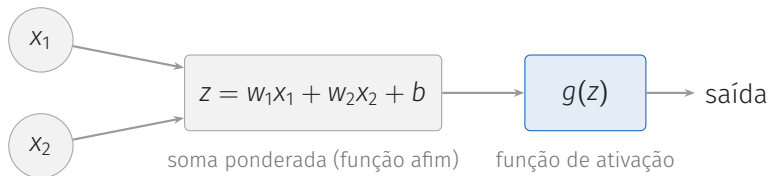


A função de ativação tem **dois trabalhos**, conforme o lugar da camada:

Nas camadas ocultas

Dobrar as retas, para a rede desenhar **curvas**. Sem ela, a rede inteira é uma reta (Aula 1).

Onde fica a função de ativação



A função de ativação tem **dois trabalhos**, conforme o lugar da camada:

Nas camadas ocultas

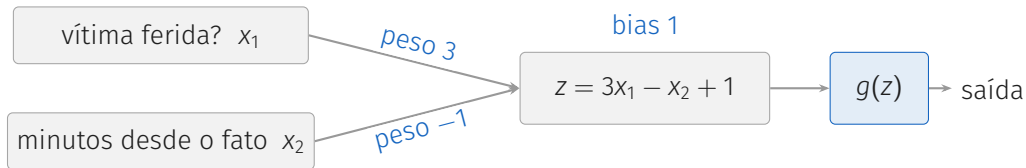
Dobrar as retas, para a rede desenhar **curvas**. Sem ela, a rede inteira é uma reta (Aula 1).

Na camada de saída

Colocar a resposta no **formato certo**: uma probabilidade, um número qualquer, uma classe entre várias.

Exemplo: um neurônio, com números

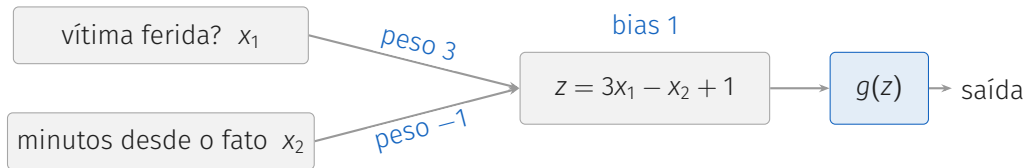
A chamada ao 190 é urgente? Um neurônio com duas entradas:



Chamada	x_1 (ferida?)	x_2 (minutos)	Conta	z
A	1 (sim)	1		
B	0 (não)	3		

Exemplo: um neurônio, com números

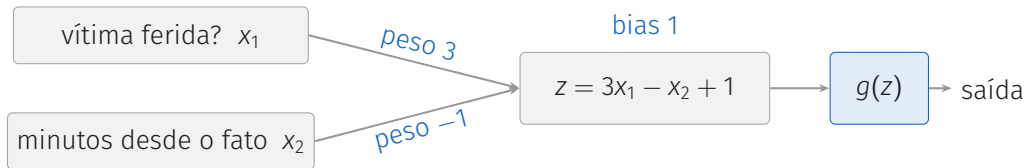
A chamada ao 190 é urgente? Um neurônio com duas entradas:



Chamada	x_1 (ferida?)	x_2 (minutos)	Conta	z
A	1 (sim)	1	$3 \cdot 1 - 1 + 1$	3
B	0 (não)	3		

Exemplo: um neurônio, com números

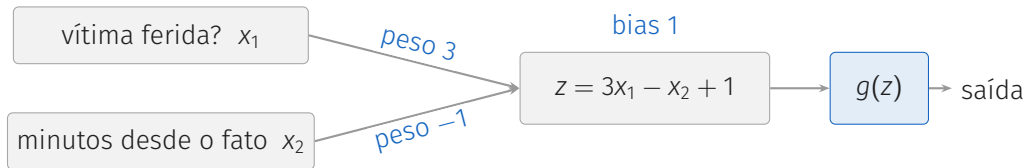
A chamada ao 190 é urgente? Um neurônio com duas entradas:



Chamada	x_1 (ferida?)	x_2 (minutos)	Conta	z
A	1 (sim)	1	$3 \cdot 1 - 1 + 1$	3
B	0 (não)	3	$3 \cdot 0 - 3 + 1$	-2

Exemplo: um neurônio, com números

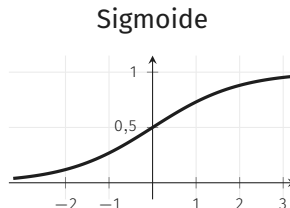
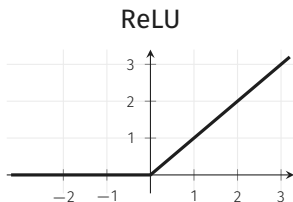
A chamada ao 190 é urgente? Um neurônio com duas entradas:



Chamada	x_1 (ferida?)	x_2 (minutos)	Conta	z
A	1 (sim)	1	$3 \cdot 1 - 1 + 1$	3
B	0 (não)	3	$3 \cdot 0 - 3 + 1$	-2

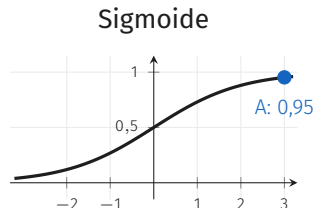
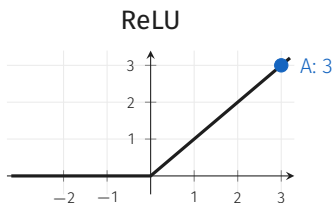
Aqui nós escolhemos os pesos e o bias. Numa rede de verdade, quem os escolhe é o treino.

Exemplo: aplicando a função de ativação



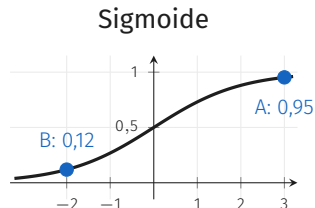
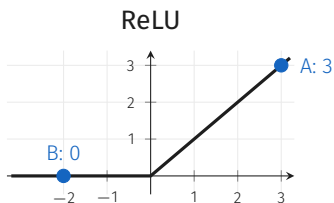
Chamada	z	Nenhuma	ReLU	Sigmoide
A	3			
B	-2			

Exemplo: aplicando a função de ativação



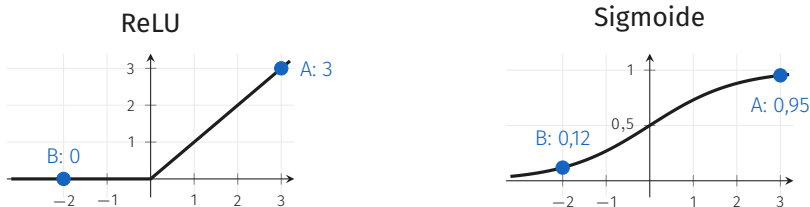
Chamada	z	Nenhuma	ReLU	Sigmoide
A	3	3	3	0,95
B	-2			

Exemplo: aplicando a função de ativação



Chamada	z	Nenhuma	ReLU	Sigmoide
A	3	3	3	0,95
B	-2	-2	0	0,12

Exemplo: aplicando a função de ativação



Chamada	z	Nenhuma	ReLU	Sigmoide
A	3	3	3	0,95
B	-2	-2	0	0,12

- **ReLU:** negativo vira 0. O z de B era -2: para a chamada B, esse neurônio “desliga”;
- **Sigmoide:** o z vira uma probabilidade. A: 95% de chance de ser urgente; B: 12%. O valor se lê no gráfico: ninguém calcula de cabeça.

Agora vocês: chamadas C e D

O mesmo neurônio: $z = 3x_1 - x_2 + 1$. 1) Calculem z . 2) ReLU: negativo vira 0; positivo fica igual. 3) Sigmoid: procurem z na tabela.

z	-3	-2	-1	0	1	2	3
sigmoide de z	0,05	0,12	0,27	0,50	0,73	0,88	0,95

Chamada	x_1 (ferida?)	x_2 (minutos)	z	ReLU	Sigmoide
C	1 (sim)	4			
D	0 (não)	2			

Agora vocês: chamadas C e D

O mesmo neurônio: $z = 3x_1 - x_2 + 1$. 1) Calculem z . 2) ReLU: negativo vira 0; positivo fica igual. 3) Sigmoid: procurem z na tabela.

z	-3	-2	-1	0	1	2	3
sigmoide de z	0,05	0,12	0,27	0,50	0,73	0,88	0,95

Chamada	x_1 (ferida?)	x_2 (minutos)	z	ReLU	Sigmoide
C	1 (sim)	4	$3 - 4 + 1 = 0$	0	0,50
D	0 (não)	2			

Agora vocês: chamadas C e D

O mesmo neurônio: $z = 3x_1 - x_2 + 1$. 1) Calculem z . 2) ReLU: negativo vira 0; positivo fica igual. 3) Sigmoid: procurem z na tabela.

z	-3	-2	-1	0	1	2	3
sigmoide de z	0,05	0,12	0,27	0,50	0,73	0,88	0,95

Chamada	x_1 (ferida?)	x_2 (minutos)	z	ReLU	Sigmoide
C	1 (sim)	4	$3 - 4 + 1 = 0$	0	0,50
D	0 (não)	2	$0 - 2 + 1 = -1$	0	0,27

Para discutir: a chamada C

Para discutir

A chamada C tem vítima ferida, mas a sigmoide deu 0,50: “não sei”. Por quê?

Para discutir: a chamada C

Para discutir

A chamada C tem vítima ferida, mas a sigmoide deu 0,50: “não sei”. Por quê?

$$z = \underbrace{3 \cdot 1}_{\text{vítima ferida}} \underbrace{- 4}_{\text{4 minutos}} \underbrace{+ 1}_{\text{bias}} = 0$$

- Os 4 minutos (-4) anulam a vítima ferida ($+3$) e o bias ($+1$). Dá $z = 0$, e a sigmoide de 0 é 0,50;
- As entradas puxam para lados opostos, e uma pode cancelar a outra. Com 2 minutos, a mesma chamada daria $z = 2$ e sigmoide 0,88.

Pergunta para a turma

Para discutir

Vocês só têm uma régua. Como desenhar um círculo?

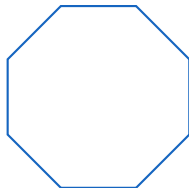
Pergunta para a turma

Para discutir

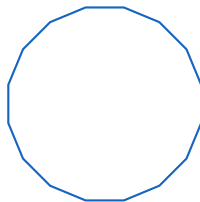
Vocês só têm uma régua. Como desenhar um círculo?



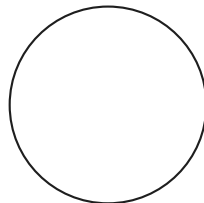
4 retas



8 retas



16 retas



círculo

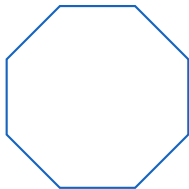
Pergunta para a turma

Para discutir

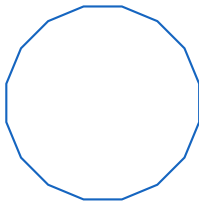
Vocês só têm uma régua. Como desenhar um círculo?



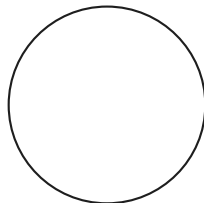
4 retas



8 retas



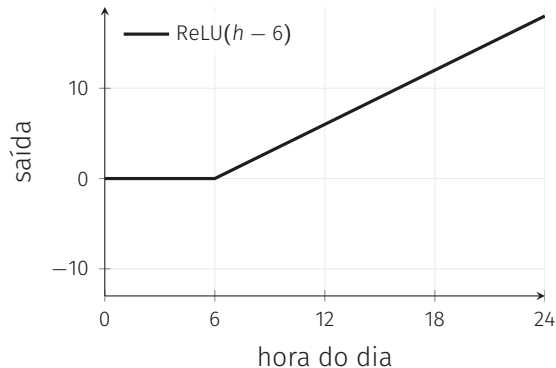
16 retas



círculo

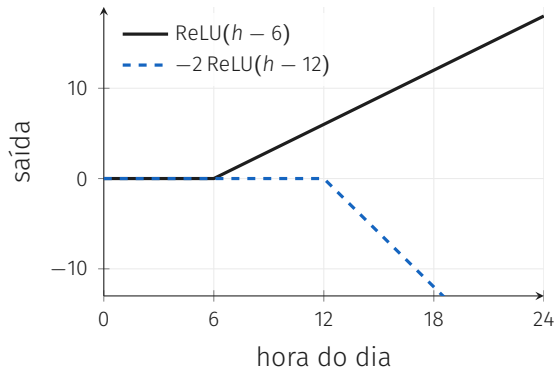
Com muitos pedaços de reta, um em seguida do outro. Cada mudança de direção é uma **dobra**. A ReLU faz exatamente isso: **cada neurônio acrescenta uma dobra**.

Um neurônio ReLU: uma dobra



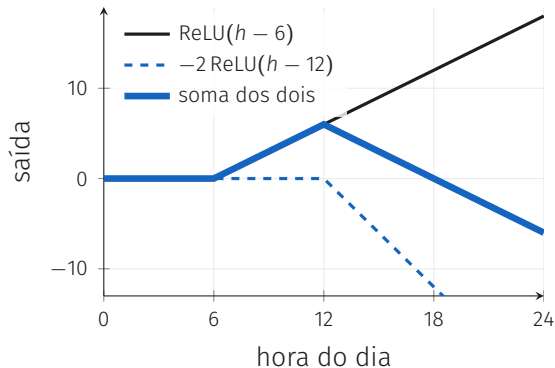
- $\text{ReLU}(h - 6)$: zero até as 6h, depois sobe. Uma dobra, **no ponto que o bias escolhe**;

Um neurônio ReLU: uma dobra



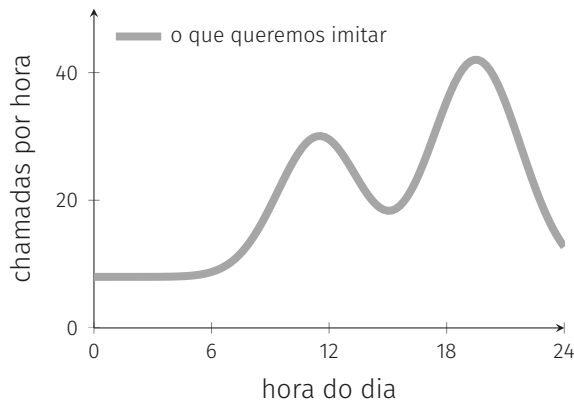
- $\text{ReLU}(h - 6)$: zero até as 6h, depois sobe. Uma dobra, **no ponto que o bias escolhe**;
- O **peso** escolhe a inclinação depois da dobra, e se ela sobe ou desce;

Um neurônio ReLU: uma dobra



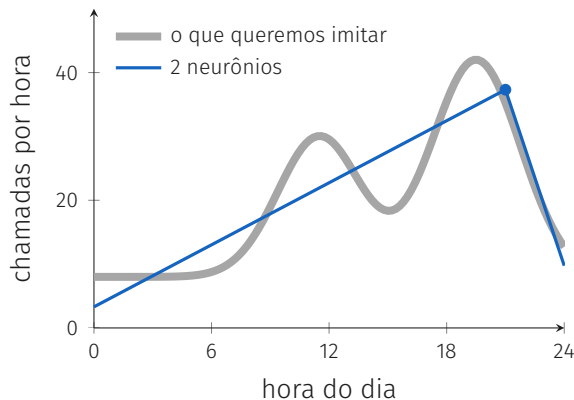
- $\text{ReLU}(h - 6)$: zero até as 6h, depois sobe. Uma dobra, **no ponto que o bias escolhe**;
- O **peso** escolhe a inclinação depois da dobra, e se ela sobe ou desce;
- Somando dois neurônios: sobe das 6h às 12h e depois desce. **Duas dobras**;
- No treino, a rede escolhe sozinha onde dobrar e quanto.

Somando dobras: chamadas ao 190 ao longo do dia



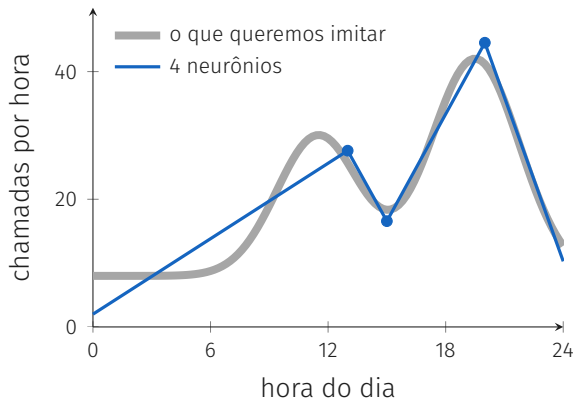
- Uma rede com 1 entrada (a hora), 1 camada oculta com ReLU e 1 saída;

Somando dobras: chamadas ao 190 ao longo do dia



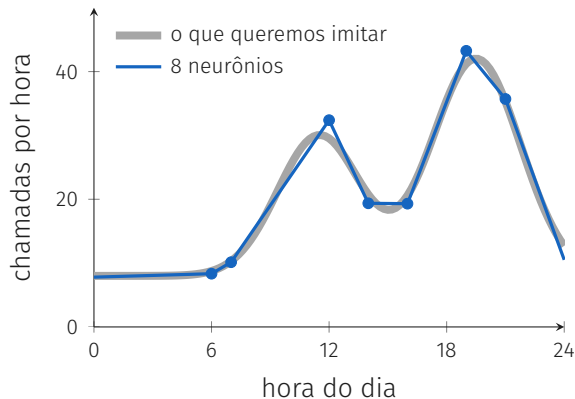
- Uma rede com 1 entrada (a hora), 1 camada oculta com ReLU e 1 saída;
- Os pontos azuis são as dobras;

Somando dobras: chamadas ao 190 ao longo do dia



- Uma rede com 1 entrada (a hora), 1 camada oculta com ReLU e 1 saída;
- Os pontos azuis são as dobras;
- Mais neurônios, mais dobras, *curva mais parecida*;

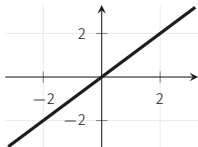
Somando dobras: chamadas ao 190 ao longo do dia



- Uma rede com 1 entrada (a hora), 1 camada oculta com ReLU e 1 saída;
- Os pontos azuis são as dobras;
- Mais neurônios, mais dobras, *curva mais parecida*;
- Com 8 neurônios, a rede já imita bem a curva. É o que o teorema da aproximação universal diz (Aula 1).

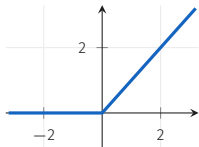
As funções de ativação mais usadas

Nenhuma (linear)



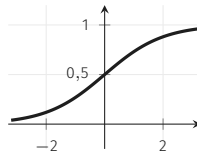
deixa passar como está
saída: qualquer número

ReLU



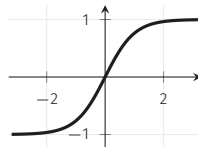
zera o negativo
saída: de 0 em diante

Sigmoide



espreme entre 0 e 1
saída: probabilidade

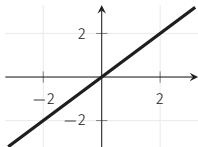
Tanh



espreme entre -1 e 1
saída: de -1 a 1

As funções de ativação mais usadas

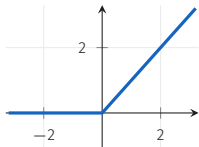
Nenhuma (linear)



deixa passar como está
saída: qualquer número

saída de regressão

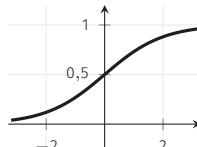
ReLU



zera o negativo
saída: de 0 em diante

camadas ocultas
(o padrão)

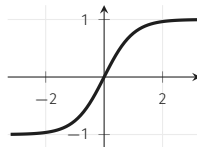
Sigmoide



espreme entre 0 e 1
saída: probabilidade

saída de classifi-
cação sim/não

Tanh

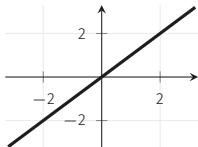


espreme entre -1 e 1
saída: de -1 a 1

camadas ocultas
de redes antigas e
das LSTM (Aula 6)

As funções de ativação mais usadas

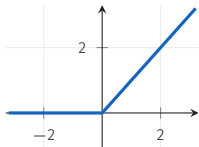
Nenhuma (linear)



deixa passar como está
saída: qualquer número

saída de regressão

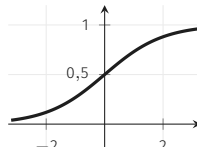
ReLU



zera o negativo
saída: de 0 em diante

camadas ocultas
(o padrão)

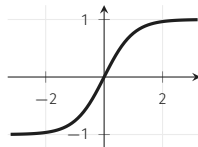
Sigmoide



espreme entre 0 e 1
saída: probabilidade

saída de classificação
sim/não

Tanh



espreme entre -1 e 1
saída: de -1 a 1

camadas ocultas
de redes antigas e
das LSTM (Aula 6)

E a **softmax**, para escolher uma classe entre várias, que veremos daqui a pouco.

Completem a tabela

Usando os gráficos do slide anterior. z é o valor que entra na função de ativação.

z	-3	-1	0	1	3
Nenhuma (linear)					
ReLU					
Sigmoide					
Tanh					

Completem a tabela

Usando os gráficos do slide anterior. z é o valor que entra na função de ativação.

z	-3	-1	0	1	3
Nenhuma (linear)	-3	-1	0	1	3
ReLU					
Sigmoide					
Tanh					

Completem a tabela

Usando os gráficos do slide anterior. z é o valor que entra na função de ativação.

z	-3	-1	0	1	3
Nenhuma (linear)	-3	-1	0	1	3
ReLU	0	0	0	1	3
Sigmoide					
Tanh					

Completem a tabela

Usando os gráficos do slide anterior. z é o valor que entra na função de ativação.

z	-3	-1	0	1	3
Nenhuma (linear)	-3	-1	0	1	3
ReLU	0	0	0	1	3
Sigmoide	0,05	0,27	0,50	0,73	0,95
Tanh					

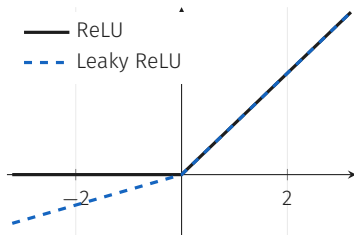
Completem a tabela

Usando os gráficos do slide anterior. z é o valor que entra na função de ativação.

z	-3	-1	0	1	3
Nenhuma (linear)	-3	-1	0	1	3
ReLU	0	0	0	1	3
Sigmoide	0,05	0,27	0,50	0,73	0,95
Tanh	-1,00	-0,76	0	0,76	1,00

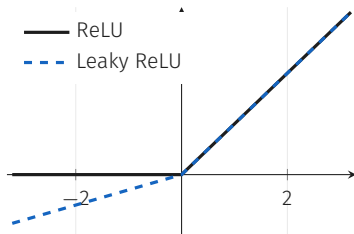
- A sigmoide nunca chega a 0 nem a 1: só se aproxima. Por isso serve como probabilidade;
- Em $z = 0$, a sigmoide dá 0,5: “não sei”.

ReLU: cuidados e variantes



- **Neurônio morto:** se um neurônio ReLU só recebe valores negativos, a saída é sempre 0, a inclinação é 0, e ele para de aprender;
- Acontece mais com taxa de aprendizado alta;

ReLU: cuidados e variantes



- **Neurônio morto:** se um neurônio ReLU só recebe valores negativos, a saída é sempre 0, a inclinação é 0, e ele para de aprender;
- Acontece mais com taxa de aprendizado alta;
- **Leaky ReLU:** deixa passar um pouco do negativo, e o neurônio não morre;
- **GELU:** uma ReLU suavizada, usada nos Transformers (Aula 7);
- Na prática: comece com *"relu"*.

A ativação da saída: o formato da resposta

A resposta é...	Faixa	Ativação	Exemplo
um número qualquer	de $-\infty$ a $+\infty$	nenhuma	variação de furtos no mês (%)

A ativação da saída: o formato da resposta

A resposta é...	Faixa	Ativação	Exemplo
um número qualquer	de $-\infty$ a $+\infty$	nenhuma	variação de furtos no mês (%)
a probabilidade de um “sim”	de 0 a 1	sigmoide	a chamada é trote?

A ativação da saída: o formato da resposta

A resposta é...	Faixa	Ativação	Exemplo
um número qualquer	de $-\infty$ a $+\infty$	nenhuma	variação de furtos no mês (%)
a probabilidade de um “sim”	de 0 a 1	sigmoide	a chamada é trote?
uma classe entre várias	probabilidades que somam 1	softmax	furto, roubo ou outro?

A ativação da saída: o formato da resposta

A resposta é...	Faixa	Ativação	Exemplo
um número qualquer	de $-\infty$ a $+\infty$	nenhuma	variação de furtos no mês (%)
a probabilidade de um “sim”	de 0 a 1	sigmoide	a chamada é trote?
uma classe entre várias	probabilidades que somam 1	softmax	furto, roubo ou outro?
várias etiquetas ao mesmo tempo	cada uma de 0 a 1	uma sigmoide por etiqueta	na imagem há pessoa? veículo? moto?

A ativação da saída: o formato da resposta

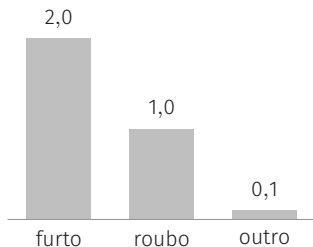
A resposta é...	Faixa	Ativação	Exemplo
um número qualquer	de $-\infty$ a $+\infty$	nenhuma	variação de furtos no mês (%)
a probabilidade de um “sim”	de 0 a 1	sigmoide	a chamada é trote?
uma classe entre várias	probabilidades que somam 1	softmax	furto, roubo ou outro?
várias etiquetas ao mesmo tempo	cada uma de 0 a 1	uma sigmoide por etiqueta	na imagem há pessoa? veículo? moto?

Regra

A ativação da saída não é escolhida pelo desempenho, e sim pelo **tipo de resposta**.

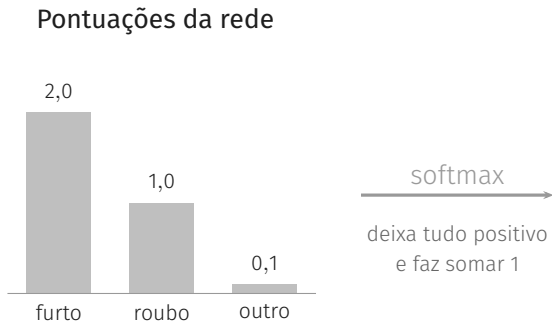
Softmax, passo a passo

Pontuações da rede



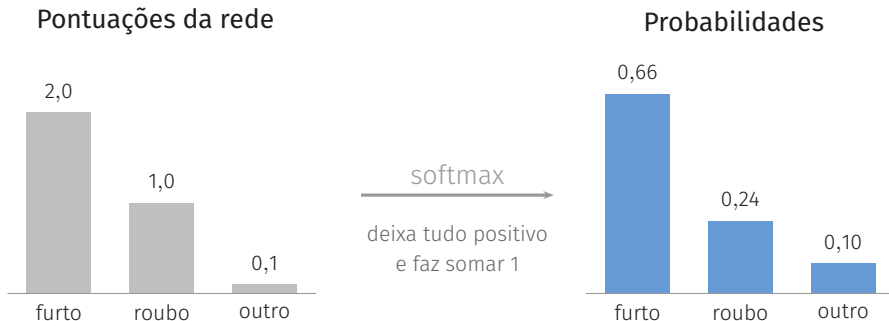
- A última camada tem um neurônio por classe: `Dense(3, activation="softmax");`

Softmax, passo a passo



- A última camada tem um neurônio por classe: `Dense(3, activation="softmax");`

Softmax, passo a passo



- A última camada tem **um neurônio por classe**: `Dense(3, activation="softmax")`;
- A maior pontuação vira a maior probabilidade, e a soma é sempre 1: a rede “divide” a certeza entre as classes;
- Classe prevista: a de maior probabilidade (furto).

Palpite: mexendo numa pontuação

Palpite, antes de ver o resultado

A pontuação de “furto” sobe de 2,0 para 4,0. As outras não mudam. O que acontece com as probabilidades de “roubo” e “outro”?

Palpite: mexendo numa pontuação

Palpite, antes de ver o resultado

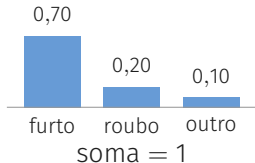
A pontuação de “furto” sobe de 2,0 para 4,0. As outras não mudam. O que acontece com as probabilidades de “roubo” e “outro”?

	furto	roubo	outro
pontuações	2,0 → 4,0	1,0	0,1
probabilidades antes	0,66	0,24	0,10
probabilidades depois	0,93	0,05	0,02

As outras **caem**, mesmo sem mudar de pontuação: como a soma é 1, quando uma classe ganha, as outras perdem. É por isso que a softmax serve para “uma classe entre várias”.

Multiclasse ou multirrótulo?

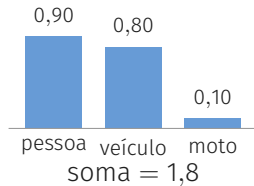
Multiclasse: **uma** classe entre várias



Cada ocorrência tem **uma** natureza.

Rótulo: [0, 1, 0] (roubo): **um só** 1.

Multirrótulo: **várias** etiquetas juntas

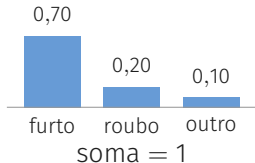


Uma imagem pode ter **várias** coisas.

Rótulo: [1, 1, 0] (pessoa e veículo): **vários** 1.

Multiclasse ou multirrótulo?

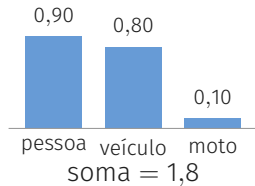
Multiclasse: **uma** classe entre várias



Cada ocorrência tem **uma** natureza.

Rótulo: [0, 1, 0] (roubo): **um só** 1.

Multirrótulo: **várias** etiquetas juntas



Uma imagem pode ter **várias** coisas.

Rótulo: [1, 1, 0] (pessoa e veículo): **vários** 1.

A pergunta que decide

O mesmo exemplo pode ter **mais de uma** resposta certa? Não: multiclasse. Sim: multirrótulo.

No Keras: multiclasse e multirrótulo

Multiclasse: furto, roubo ou outro?

```
layers.Dense(3,  
             activation="softmax")  
loss="sparse_categorical_crossentropy"
```

- As 3 saídas **somam 1**;
- Rótulo: um inteiro, 0, 1 ou 2. Se vier em one-hot, [0, 1, 0], a perda é *"categorical_crossentropy"*;
- Previsão: a classe de **maior** probabilidade.

A perda (*loss*) é o assunto do próximo bloco. Por enquanto: ela anda junto com a ativação da saída.

Multirrótulo: pessoa? veículo? moto?

```
layers.Dense(3,  
             activation="sigmoid")  
loss="binary_crossentropy"
```

- Cada saída vai de 0 a 1, **independente** das outras;
- Rótulo: um vetor de 0 e 1, como [1, 1, 0];
- Previsão: **todas** as etiquetas acima de 0,5.

Multiclasse ou multirrótulo? Exemplos

O mesmo exemplo pode ter mais de uma resposta certa?

Tarefa	Tipo
1. Natureza da ocorrência: furto, roubo ou outro	
2. O que aparece na imagem da câmera: pessoa, veículo, moto	
3. Qual delegacia deve receber o caso	
4. Crimes citados no boletim: ameaça, lesão, dano...	
5. Características do suspeito no relato: boné, tatuagem, óculos	
6. Turno da ocorrência: manhã, tarde ou noite	
7. Dígito escrito à mão, de 0 a 9	
8. Gêneros de um filme: ação, comédia, drama	

Multiclasse ou multirrótulo? Exemplos

O mesmo exemplo pode ter mais de uma resposta certa?

Tarefa	Tipo
1. Natureza da ocorrência: furto, roubo ou outro	multiclasse
2. O que aparece na imagem da câmera: pessoa, veículo, moto	multirrótulo
3. Qual delegacia deve receber o caso	multiclasse
4. Crimes citados no boletim: ameaça, lesão, dano...	multirrótulo
5. Características do suspeito no relato: boné, tatuagem, óculos	multirrótulo
6. Turno da ocorrência: manhã, tarde ou noite	multiclasse
7. Dígito escrito à mão, de 0 a 9	multiclasse
8. Gêneros de um filme: ação, comédia, drama	multirrótulo

Multiclasse ou multirrótulo? Exemplos

O mesmo exemplo pode ter mais de uma resposta certa?

Tarefa	Tipo
1. Natureza da ocorrência: furto, roubo ou outro	multiclasse
2. O que aparece na imagem da câmera: pessoa, veículo, moto	multirrótulo
3. Qual delegacia deve receber o caso	multiclasse
4. Crimes citados no boletim: ameaça, lesão, dano...	multirrótulo
5. Características do suspeito no relato: boné, tatuagem, óculos	multirrótulo
6. Turno da ocorrência: manhã, tarde ou noite	multiclasse
7. Dígito escrito à mão, de 0 a 9	multiclasse
8. Gêneros de um filme: ação, comédia, drama	multirrótulo

- Multiclasse: *activation="softmax"* e *loss="sparse_categorical_crossentropy"*;
- Multirrótulo: *activation="sigmoid"* e *loss="binary_crossentropy"*.

Para discutir: assunto e idioma

Para discutir

Uma notícia pode ser, ao mesmo tempo, de “política” e de “economia”. Softmax ou sigmoides? E para classificar o idioma da notícia?

Para discutir

Uma notícia pode ser, ao mesmo tempo, de “política” e de “economia”. Softmax ou sigmóides? E para classificar o idioma da notícia?

- **Assuntos:** uma **sigmoide** por assunto. A mesma notícia pode ter 0,9 em política e 0,8 em economia;
- **Idioma:** **softmax**. Cada notícia tem um idioma só, e as probabilidades somam 1;
- A pergunta que decide: a resposta é **uma** opção entre várias, ou **várias ao mesmo tempo**?

Pergunta: ReLU na saída

Uma rede prevê a temperatura mínima de amanhã em Curitiba, que pode ser negativa. A última camada é `Dense(1, activation="relu")`. O que acontece?

- a) Nada de errado: ReLU é a ativação padrão;
- b) A rede nunca prevê temperaturas abaixo de 0 °C;
- c) O Keras dá erro, porque ReLU não pode ficar na saída;
- d) A rede só prevê 0 ou 1.

Pergunta: ReLU na saída

Uma rede prevê a temperatura mínima de amanhã em Curitiba, que pode ser negativa. A última camada é `Dense(1, activation="relu")`. O que acontece?

- a) Nada de errado: ReLU é a ativação padrão;
- b) A rede nunca prevê temperaturas abaixo de 0 °C;
- c) O Keras dá erro, porque ReLU não pode ficar na saída;
- d) A rede só prevê 0 ou 1.

Resposta: b. A ReLU zera o negativo: a saída mínima é 0. O Keras não avisa (c). ReLU é o padrão nas camadas **ocultas**; na saída de regressão, sem ativação: `Dense(1)`.

Atividade: qual ativação na saída?

Em grupos, 5 min. Para cada tarefa: quantos neurônios na saída, e qual ativação?

Tarefa	Saída	Neurônios
1. O veículo abordado é roubado?		
2. Minutos até a viatura chegar		
3. Qual das 5 delegacias deve receber o caso?		
4. Que crimes aparecem no relato (pode haver vários)?		
5. Variação percentual de roubos no próximo mês		
6. Dígito escrito à mão, de 0 a 9		

Atividade: qual ativação na saída?

Em grupos, 5 min. Para cada tarefa: quantos neurônios na saída, e qual ativação?

Tarefa	Saída	Neurônios
1. O veículo abordado é roubado?	sigmoide	1
2. Minutos até a viatura chegar	nenhuma	1
3. Qual das 5 delegacias deve receber o caso?	softmax	5
4. Que crimes aparecem no relato (pode haver vários)?	uma sigmoide por crime	1 por crime
5. Variação percentual de roubos no próximo mês	nenhuma	1
6. Dígito escrito à mão, de 0 a 9	softmax	10

Para discutir: ReLU na saída?

Para discutir

Na tarefa 2 (minutos até a viatura chegar), o tempo nunca é negativo. Faz sentido usar ReLU na saída? O que pode dar errado?

Para discutir: ReLU na saída?

Para discutir

Na tarefa 2 (minutos até a viatura chegar), o tempo nunca é negativo. Faz sentido usar ReLU na saída? O que pode dar errado?

- **Pode funcionar:** a ReLU nunca dá negativo, e o tempo também nunca é negativo;
- **O risco:** se o neurônio de saída só receber valores negativos, ele “morre”. A rede passa a prever 0 minutos para todas as chamadas, e não sai mais disso;
- **Na prática:** sem ativação costuma bastar. Se aparecer uma previsão negativa, troca-se por 0 na hora de usar.

Camadas ocultas

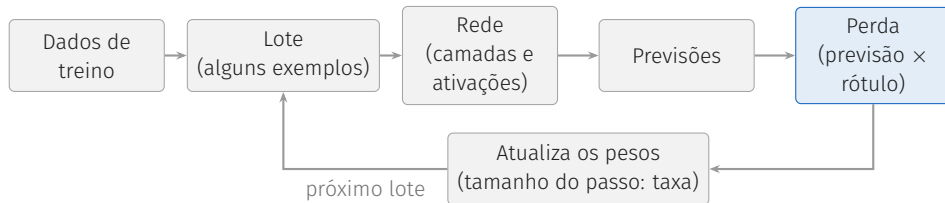
- Servem para **dobrar**: sem elas, só retas;
- Use *"relu"*.

Camada de saída

- Servem para dar o **formato** da resposta;
- Número: nenhuma;
- Sim ou não: *"sigmoid"*;
- Uma classe entre K : *"softmax"*, K neurônios;
- Várias etiquetas: K sigmóides.

Funções de perda

A função de perda: a régua do treino



todos os lotes uma vez = 1 **época**; o treino repete por várias épocas

- A perda resume o erro em **um número**: quanto menor, melhor;
- O treino só enxerga esse número: **o que a perda não mede, a rede não aprende.**

Para discutir: a prova e a perda

Para discutir

Se a prova valesse só a resposta final, e não o raciocínio, como vocês estudariam? O que isso tem a ver com a perda?

Para discutir: a prova e a perda

Para discutir

Se a prova valesse só a resposta final, e não o raciocínio, como vocês estudariam? O que isso tem a ver com a perda?

- Vocês decorariam respostas, em vez de entender o raciocínio: a gente estuda para o que a prova cobra;
- A rede faz o mesmo: aprende **exatamente** o que a perda mede, e nada além disso;
- Se a perda não cobra um tipo de erro, a rede não evita esse erro. Escolher a perda é dizer à rede **quais erros importam**.

Regressão: um exemplo com números

Tempo até a viatura chegar, em minutos, em 5 ocorrências.

Ocorrência	Real	Previsto	Erro
1	12	10	2
2	8	9	-1
3	15	14	1
4	10	10	0
5	9	19	-10
média:			

Regressão: um exemplo com números

Tempo até a viatura chegar, em minutos, em 5 ocorrências.

Ocorrência	Real	Previsto	Erro	Erro
1	12	10	2	2
2	8	9	-1	1
3	15	14	1	1
4	10	10	0	0
5	9	19	-10	10
média:				2,8 (MAE)

- **MAE** (erro absoluto médio): a média dos erros sem o sinal. “Em média, erra 2,8 minutos”;

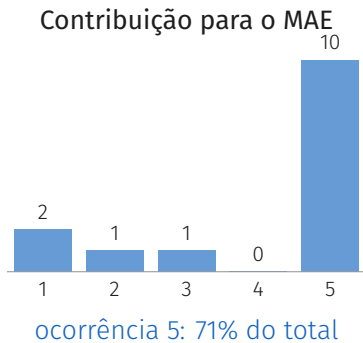
Regressão: um exemplo com números

Tempo até a viatura chegar, em minutos, em 5 ocorrências.

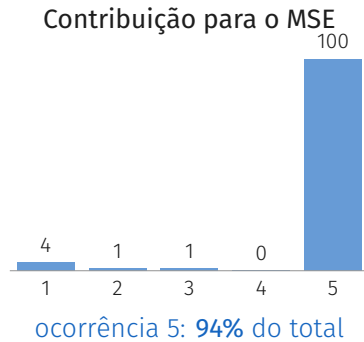
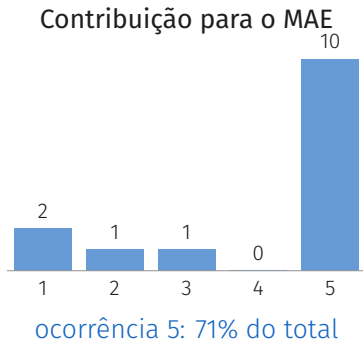
Ocorrência	Real	Previsto	Erro	Erro	Erro ²
1	12	10	2	2	4
2	8	9	-1	1	1
3	15	14	1	1	1
4	10	10	0	0	0
5	9	19	-10	10	100
			média:	2,8 (MAE)	21,2 (MSE)

- **MAE** (erro absoluto médio): a média dos erros sem o sinal. “Em média, erra 2,8 minutos”;
- **MSE** (erro quadrático médio): a média dos erros ao quadrado.

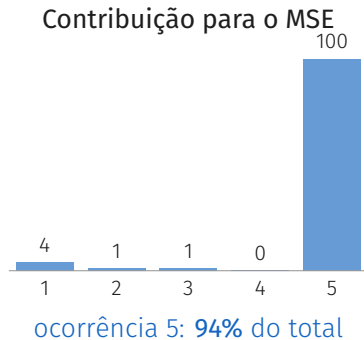
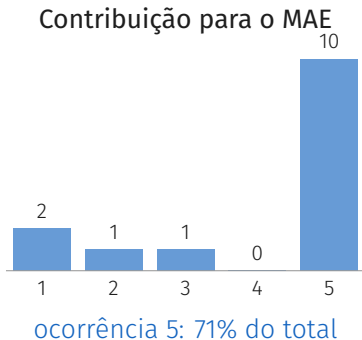
Desenhando o erro



Desenhando o erro

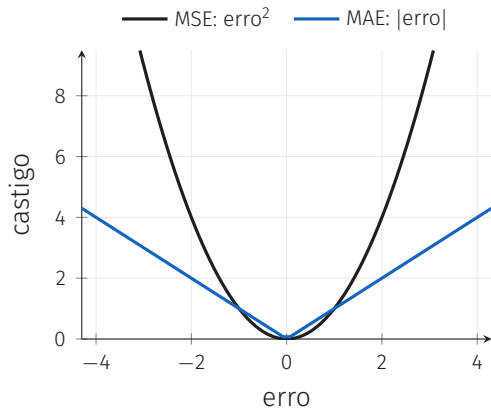


Desenhando o erro



- Com o MSE, o treino vai se preocupar quase só com a ocorrência 5;
- O quadrado faz o erro grande pesar **muito mais** que vários erros pequenos.

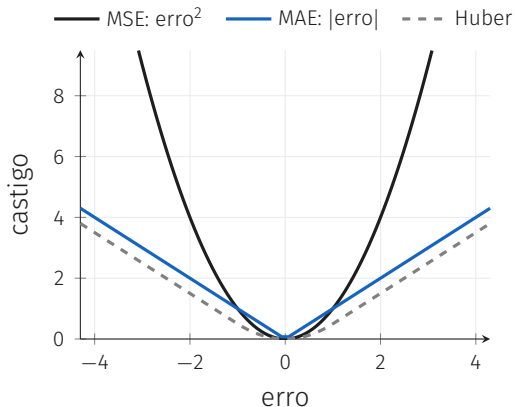
O castigo de cada perda



Erro	MSE	MAE
1	1	1
2	4	2
5	25	5
10	100	10

- No MSE, errar por 10 é **100 vezes** pior que errar por 1. No MAE, 10 vezes;

O castigo de cada perda



Erro	MSE	MAE
1	1	1
2	4	2
5	25	5
10	100	10

- No MSE, errar por 10 é **100 vezes** pior que errar por 1. No MAE, 10 vezes;
- **Huber**: quadrado para erros pequenos, reta para os grandes. Um meio-termo.

Qual modelo vocês preferem?

Dois modelos preveem o tempo de chegada em 5 ocorrências. Erros, em minutos:

	1	2	3	4	5	
Modelo A	3	3	3	3	3	
Modelo B	0	0	0	0	15	

Qual modelo vocês preferem?

Dois modelos preveem o tempo de chegada em 5 ocorrências. Erros, em minutos:

	1	2	3	4	5	
Modelo A	3	3	3	3	3	
Modelo B	0	0	0	0	15	

Para discutir

1. Para o despacho de uma ambulância, qual vocês preferem?
2. E para estimar o horário de chegada de uma entrega de material de escritório?

Qual modelo vocês preferem?

Dois modelos preveem o tempo de chegada em 5 ocorrências. Erros, em minutos:

	1	2	3	4	5	MAE	MSE
Modelo A	3	3	3	3	3	3	9
Modelo B	0	0	0	0	15	3	45

Para discutir

1. Para o despacho de uma ambulância, qual vocês preferem?
2. E para estimar o horário de chegada de uma entrega de material de escritório?

Ambulância: A. Um atraso de 15 minutos pode custar uma vida, e o MSE pune esse erro.

Entrega: B serve, porque errar uma entrega não é grave; o MAE vê os dois iguais. A perda diz à rede **quais erros são mais graves**.

Regressão: quando usar cada perda

Perda	No Keras	Quando usar
MSE	<code>loss="mse"</code>	O padrão. Erros grandes são graves e devem pesar mais.
MAE	<code>loss="mae"</code>	Há valores atípicos ou erros de registro que não devem dominar o treino.
Huber	<code>loss=keras.losses.Huber()</code>	Meio-termo: se comporta como o MSE nos erros pequenos e como o MAE nos grandes.

- A perda do treino e o número que vocês relatam podem ser diferentes: treinar com MSE e relatar o MAE (“erra 2,8 minutos em média”) é comum;
- Para isso: `metrics=["mae"]`.

Palpite: erros de digitação

Palpite, antes de ver o resultado

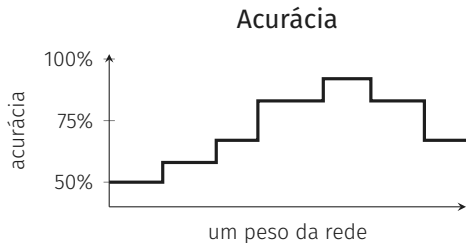
Numa base de tempos de deslocamento, 1% dos registros tem erro de digitação: 300 minutos em vez de 30. A rede é treinada com MSE. O que acontece com as previsões das ocorrências **normais**?

Palpite, antes de ver o resultado

Numa base de tempos de deslocamento, 1% dos registros tem erro de digitação: 300 minutos em vez de 30. A rede é treinada com MSE. O que acontece com as previsões das ocorrências **normais**?

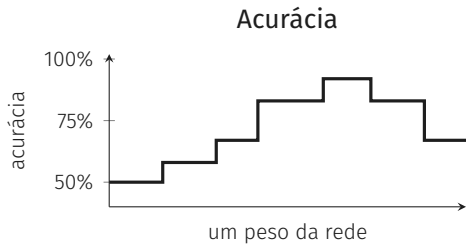
- Um erro de 270 minutos, ao quadrado, vale 72.900. Poucos registros assim dominam a perda;
- A rede desloca as previsões para cima, tentando reduzir esses erros enormes, e **erra mais nas ocorrências normais**;
- Com MAE ou Huber, o efeito é bem menor;
- Melhor ainda: **olhar os dados** e corrigir os registros.

Classificação: por que não usar a acurácia como perda?



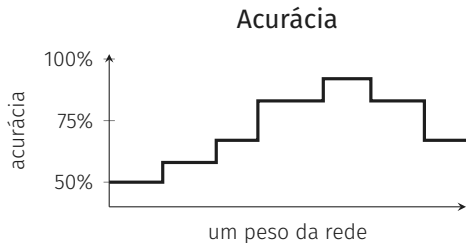
- A acurácia muda **aos saltos**: mexer um pouco num peso quase nunca muda um acerto;
- Num degrau plano não há inclinação, e a descida do gradiente **não sabe para onde ir**;

Classificação: por que não usar a acurácia como perda?



- A acurácia muda **aos saltos**: mexer um pouco num peso quase nunca muda um acerto;
- Num degrau plano não há inclinação, e a descida do gradiente **não sabe para onde ir**;
- A entropia cruzada muda **suavemente**: cada pequena mudança nos pesos aparece na perda;

Classificação: por que não usar a acurácia como perda?



- A acurácia muda **aos saltos**: mexer um pouco num peso quase nunca muda um acerto;
- Num degrau plano não há inclinação, e a descida do gradiente **não sabe para onde ir**;
- A entropia cruzada muda **suavemente**: cada pequena mudança nos pesos aparece na perda;
- E a acurácia não diferencia uma rede que acertou com 51% de certeza de outra que acertou com 99%.

Dois modelos com 100% de acurácia

Três chamadas, todas trote (rótulo 1). Probabilidade de trote que cada modelo deu:

	chamada 1	chamada 2	chamada 3	acurácia
Modelo A	0,51	0,55	0,52	100%
Modelo B	0,95	0,90	0,99	100%

Dois modelos com 100% de acurácia

Três chamadas, todas trote (rótulo 1). Probabilidade de trote que cada modelo deu:

	chamada 1	chamada 2	chamada 3	acurácia
Modelo A	0,51	0,55	0,52	100%
Modelo B	0,95	0,90	0,99	100%

Para discutir

Os dois acertaram tudo. Em qual vocês confiariam mais numa chamada nova? Por quê?

Dois modelos com 100% de acurácia

Três chamadas, todas trote (rótulo 1). Probabilidade de trote que cada modelo deu:

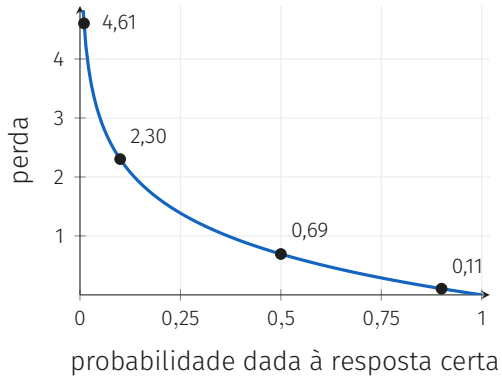
	chamada 1	chamada 2	chamada 3	acurácia	perda
Modelo A	0,51	0,55	0,52	100%	0,64
Modelo B	0,95	0,90	0,99	100%	0,06

Para discutir

Os dois acertaram tudo. Em qual vocês confiariam mais numa chamada nova? Por quê?

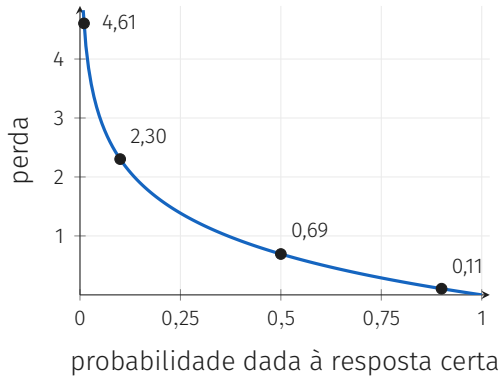
No Modelo B: ele acerta com folga. O Modelo A acerta “por pouco”, com 51% a 55%: uma pequena mudança nos dados e ele erra. A acurácia não vê a diferença; a entropia cruzada (perda) vê: 0,64 contra 0,06.

Entropia cruzada: o tamanho da surpresa



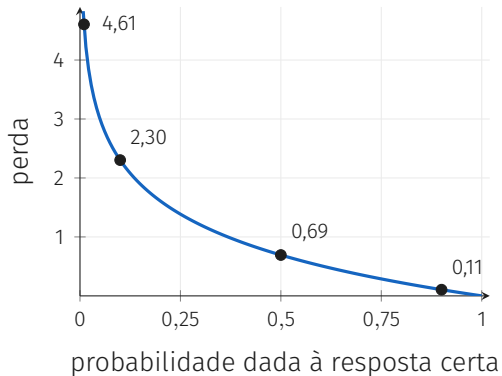
- A perda olha só a probabilidade que a rede deu à **resposta certa**;
- Perto de 1: perda perto de 0. Perto de 0: a perda **dispara**;

Entropia cruzada: o tamanho da surpresa



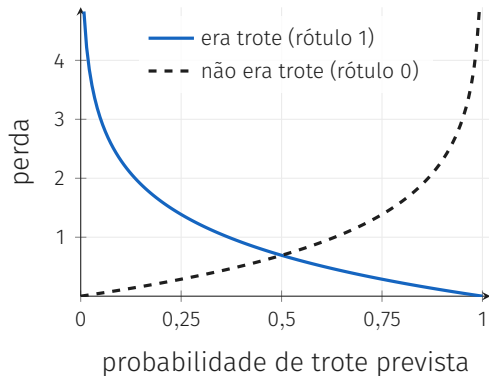
- A perda olha só a probabilidade que a rede deu à **resposta certa**;
- Perto de 1: perda perto de 0. Perto de 0: a perda **dispara**;
- A previsão do tempo: “90% de chance de chuva”, e choveu: pouca surpresa (0,11). “1% de chance”, e choveu: surpresa enorme (4,61);

Entropia cruzada: o tamanho da surpresa



- A perda olha só a probabilidade que a rede deu à **resposta certa**;
- Perto de 1: perda perto de 0. Perto de 0: a perda **dispara**;
- A previsão do tempo: “90% de chance de chuva”, e choveu: pouca surpresa (0,11). “1% de chance”, e choveu: surpresa enorme (4,61);
- **Errar com muita certeza custa muito caro.**

Entropia cruzada binária: rótulo 1 e rótulo 0



Rótulo	Previsto	Perda
1	0,9	0,11
1	0,2	1,61
0	0,1	0,11
0	0,9	2,30

- Perda baixa quando a probabilidade fica **perto do rótulo**;
- No Keras:
loss="binary_crossentropy",
com sigmoide na saída.

Pergunta: qual previsão tem a maior perda?

Uma rede dá a probabilidade de a chamada ser trote. Qual previsão tem a **maior** perda de entropia cruzada?

- a) Era trote; a rede deu 0,8;
- b) Não era trote; a rede deu 0,3;
- c) Era trote; a rede deu 0,05;
- d) Não era trote; a rede deu 0,5.

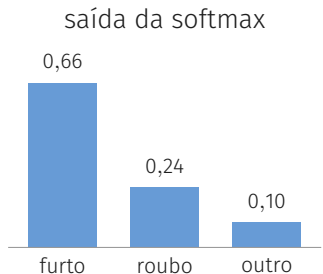
Pergunta: qual previsão tem a maior perda?

Uma rede dá a probabilidade de a chamada ser trote. Qual previsão tem a **maior** perda de entropia cruzada?

- a) Era trote; a rede deu 0,8;
- b) Não era trote; a rede deu 0,3;
- c) Era trote; a rede deu 0,05;
- d) Não era trote; a rede deu 0,5.

Resposta: c. A rede deu só 5% à resposta certa: perda 3,0. As outras: (a) 0,22; (b) a resposta certa recebeu 0,7, perda 0,36; (d) 0,69, a perda de quem diz “não sei”.

Várias classes: entropia cruzada categórica



Classe certa	Probabilidade dela	Perda
furto	0,66	0,42
roubo	0,24	1,43
outro	0,10	2,30

- A mesma ideia: só importa a probabilidade da **classe certa**;
- É a mesma curva do slide da surpresa;
- Com softmax na saída.

Rótulos como inteiros ou como vetores

Classe	Inteiro	One-hot
furto	0	[1, 0, 0]
roubo	1	[0, 1, 0]
outro	2	[0, 0, 1]

- O rótulo pode vir como um **número inteiro** ou como um **vetor one-hot**: um 1 na posição da classe, 0 no resto;
- A conta da perda é a mesma. Muda só o **formato** do rótulo.

Rótulos como inteiros ou como vetores

Classe	Inteiro	One-hot
furto	0	[1, 0, 0]
roubo	1	[0, 1, 0]
outro	2	[0, 0, 1]

- O rótulo pode vir como um **número inteiro** ou como um **vetor one-hot**: um 1 na posição da classe, 0 no resto;
- A conta da perda é a mesma. Muda só o **formato** do rótulo.

Rótulos	<i>loss=</i>
inteiros: 0, 1, 2	<i>"sparse_categorical_crossentropy"</i>
one-hot: [1, 0, 0], ...	<i>"categorical_crossentropy"</i>

Trocar uma pela outra dá erro de formato (*shape*). É um dos erros mais comuns no Keras.

```
modelo.compile(optimizer="adam",  
               loss="binary_crossentropy", # o treino minimiza  
               metrics=["accuracy"])      # nos lemos
```

Perda: para o otimizador

- Precisa mudar suavemente com os pesos;
- Entropia cruzada, MSE, MAE.

Métrica: para as pessoas

- Pode mudar aos saltos;
- Acurácia, MAE em minutos, recall.

Para discutir: o erro mais grave

Para discutir

Um modelo prioriza chamadas de violência doméstica. O que é pior: um alarme falso, ou um caso real que passa despercebido? Que número vocês acompanhariam?

Para discutir: o erro mais grave

Para discutir

Um modelo prioriza chamadas de violência doméstica. O que é pior: um alarme falso, ou um caso real que passa despercebido? Que número vocês acompanhariam?

- Pior: o **caso real que passa despercebido**. Um alarme falso custa uma viagem da viatura; um caso perdido pode custar uma vida;
- O número: o **recall**. De todos os casos reais, quantos o modelo encontrou;
- Se os casos reais são raros, a acurácia engana: um modelo que nunca aponta violência acerta quase sempre, e não encontra nenhum caso. Voltamos a isso na Aula 5.

A tabela para o Keras

Tarefa	Última camada	loss=
Regressão minutos até a viatura chegar	<code>Dense(1)</code>	<code>"mse"</code> ou <code>"mae"</code>
Binária a chamada é trote?	<code>Dense(1, activation="sigmoid")</code>	<code>"binary_crossentropy"</code>
Multiclasse furto, roubo ou outro?	<code>Dense(3, activation="softmax")</code>	<code>"sparse_categorical_crossentropy"</code>
Multirrótulo pessoa? veículo? moto?	<code>Dense(3, activation="sigmoid")</code>	<code>"binary_crossentropy"</code>

A ativação da saída e a perda andam juntas

Decidam primeiro o **tipo de resposta**. A última camada e a perda saem da tabela.

Ache o erro (1/2)

Cada trecho roda (ou quase), mas tem um problema. Qual?

1. Minutos até a viatura chegar:

```
| layers.Dense(1, activation="sigmoid") # loss="mse"
```

2. A chamada é trote?

```
| layers.Dense(1, activation="softmax") # loss="binary_crossentropy"
```

3. Furto, roubo ou outro, com rótulos 0, 1, 2:

```
| layers.Dense(3, activation="softmax") # loss="categorical_crossentropy"
```

Ache o erro (1/2)

Cada trecho roda (ou quase), mas tem um problema. Qual?

1. Minutos até a viatura chegar:

```
| layers.Dense(1, activation="sigmoid") # loss="mse"
```

A saída fica entre 0 e 1 minuto. Corrigir: **Dense(1)**, sem ativação.

2. A chamada é trote?

```
| layers.Dense(1, activation="softmax") # loss="binary_crossentropy"
```

3. Furto, roubo ou outro, com rótulos 0, 1, 2:

```
| layers.Dense(3, activation="softmax") # loss="categorical_crossentropy"
```

Ache o erro (1/2)

Cada trecho roda (ou quase), mas tem um problema. Qual?

1. Minutos até a viatura chegar:

```
| layers.Dense(1, activation="sigmoid") # loss="mse"
```

A saída fica entre 0 e 1 minuto. Corrigir: **Dense(1)**, sem ativação.

2. A chamada é trote?

```
| layers.Dense(1, activation="softmax") # loss="binary_crossentropy"
```

Softmax de um número só é sempre 1: a rede diz “trote” para tudo e não aprende.
Corrigir: **"sigmoid"**.

3. Furto, roubo ou outro, com rótulos 0, 1, 2:

```
| layers.Dense(3, activation="softmax") # loss="categorical_crossentropy"
```

Ache o erro (1/2)

Cada trecho roda (ou quase), mas tem um problema. Qual?

1. Minutos até a viatura chegar:

```
| layers.Dense(1, activation="sigmoid") # loss="mse"
```

A saída fica entre 0 e 1 minuto. Corrigir: **Dense(1)**, sem ativação.

2. A chamada é trote?

```
| layers.Dense(1, activation="softmax") # loss="binary_crossentropy"
```

Softmax de um número só é sempre 1: a rede diz “trote” para tudo e não aprende.
Corrigir: **"sigmoid"**.

3. Furto, roubo ou outro, com rótulos 0, 1, 2:

```
| layers.Dense(3, activation="softmax") # loss="categorical_crossentropy"
```

Erro de formato: os rótulos são inteiros. Corrigir:
"sparse_categorical_crossentropy".

Ache o erro (2/2)

4. Na imagem da câmera: há pessoa? veículo? moto? (pode haver várias)

```
| layers.Dense(3, activation="softmax") # loss="categorical_crossentropy"
```

5. Furto, roubo ou outro, com rótulos 0, 1, 2:

```
| layers.Dense(1) # loss="mse"
```

Ache o erro (2/2)

4. Na imagem da câmera: há pessoa? veículo? moto? (pode haver várias)

```
| layers.Dense(3, activation="softmax")    # loss="categorical_crossentropy"
```

A softmax obriga a soma a ser 1: se há pessoa e veículo, os dois não podem ter 0,9.
Corrigir: `Dense(3, activation="sigmoid")` e `"binary_crossentropy"`.

5. Furto, roubo ou outro, com rótulos 0, 1, 2:

```
| layers.Dense(1)                          # loss="mse"
```

Ache o erro (2/2)

4. Na imagem da câmera: há pessoa? veículo? moto? (pode haver várias)

```
| layers.Dense(3, activation="softmax") # loss="categorical_crossentropy"
```

A softmax obriga a soma a ser 1: se há pessoa e veículo, os dois não podem ter 0,9.
Corrigir: `Dense(3, activation="sigmoid")` e `"binary_crossentropy"`.

5. Furto, roubo ou outro, com rótulos 0, 1, 2:

```
| layers.Dense(1) # loss="mse"
```

Trata as classes como números: “roubo” fica entre “furto” e “outro”, e uma previsão de 1,4 não é classe nenhuma. Corrigir: `Dense(3, activation="softmax")` e `"sparse_categorical_crossentropy"`.

- A perda é a **régua** do treino: a rede aprende o que ela mede;
- **Regressão**: MSE (padrão; erros grandes pesam muito), MAE (resiste a valores atípicos), Huber (meio-termo);
- **Classificação**: entropia cruzada, a “surpresa” com a resposta certa. Binária com sigmoide; categórica com softmax;
- Rótulos inteiros: ***sparse_***. Rótulos one-hot: sem ***sparse_***;
- A **métrica** é para nós; a **perda** é para o otimizador.

Taxa de aprendizado

Descendo a montanha no escuro



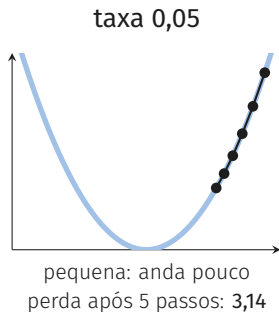
- No escuro, vocês só sentem a **inclinação** do chão sob os pés;
- O gradiente é essa inclinação: diz para que lado a perda cresce mais;
- A **taxa de aprendizado** é o tamanho do passo:

$$\text{novo peso} = \text{peso} - \text{taxa} \times \text{inclinação}$$

Para discutir

O que dá errado com passos muito pequenos? E com passos muito grandes?

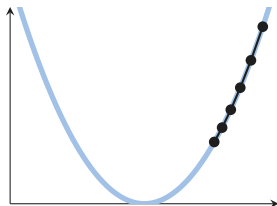
Três taxas, o mesmo ponto de partida



- Perda = peso^2 , começando no peso 3. Cada ponto é um passo;

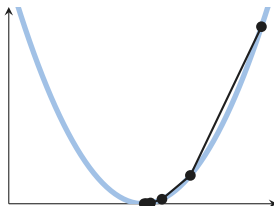
Três taxas, o mesmo ponto de partida

taxa 0,05



pequena: anda pouco
perda após 5 passos: 3,14

taxa 0,3

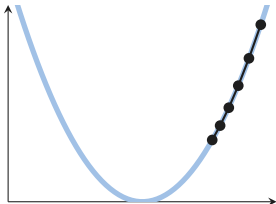


boa: chega rápido
perda após 5 passos: 0,00

- Perda = peso^2 , começando no peso 3. Cada ponto é um passo;

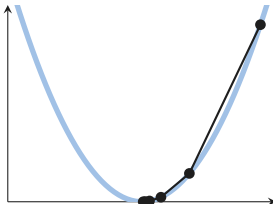
Três taxas, o mesmo ponto de partida

taxa 0,05



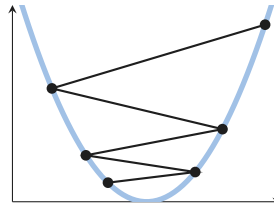
pequena: anda pouco
perda após 5 passos: **3,14**

taxa 0,3



boa: chega rápido
perda após 5 passos: **0,00**

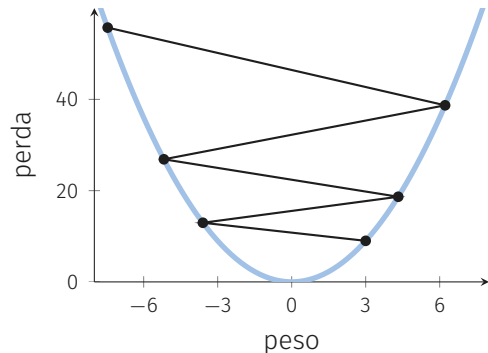
taxa 0,9



grande: zigue-zague
perda após 5 passos: **0,97**

- Perda = peso^2 , começando no peso 3. Cada ponto é um passo;
- Com a taxa grande, o passo passa do mínimo e cai do outro lado do vale.

Taxa grande demais: a perda explode

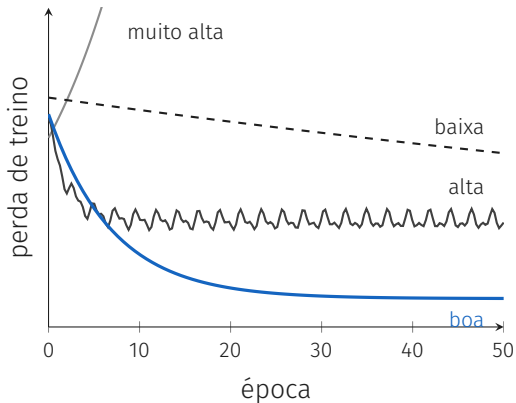


Taxa 1,1:

Passo	Peso	Perda
0	3,00	9,0
1	-3,60	13,0
2	4,32	18,7
3	-5,18	26,9
4	6,22	38,7
5	-7,46	55,7

- Cada passo cai mais longe do mínimo;
- No Keras, aparece **loss: nan**: o número ficou grande demais para o computador.

Lendo a taxa na curva de perda



- **Muito alta:** a perda sobe e vira *nan*;
- **Alta:** cai rápido no começo, depois fica pulando num valor alto;
- **Baixa:** cai quase em linha reta, devagar;
- **Boa:** cai rápido e vai se acomodando num valor baixo.

Pergunta: a perda parada

Uma rede de classificação binária treina por 50 épocas. A perda de treino fica assim: 0,693, 0,693, 0,692, 0,693, 0,692... O que é mais provável?

- a) Sobreajuste;
- b) A rede não está aprendendo: taxa baixa demais, ou a rede travada;
- c) A rede já chegou ao melhor resultado possível;
- d) O lote é grande demais.

Pergunta: a perda parada

Uma rede de classificação binária treina por 50 épocas. A perda de treino fica assim: 0,693, 0,693, 0,692, 0,693, 0,692... O que é mais provável?

- a) Sobreajuste;
- b) A rede não está aprendendo: taxa baixa demais, ou a rede travada;
- c) A rede já chegou ao melhor resultado possível;
- d) O lote é grande demais.

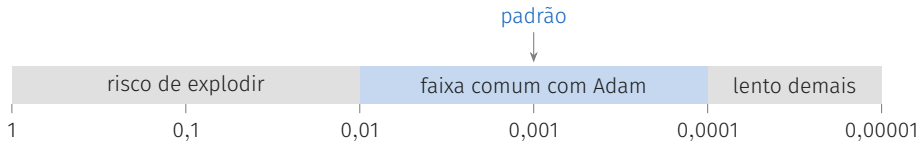
Resposta: b. 0,693 é a perda de quem responde 0,5 para tudo: “não sei”. A rede não saiu do lugar. Sobreajuste (a) apareceria na validação, com o treino caindo. Tentar: taxa 10 vezes maior, e conferir os dados e a padronização.

Como mudar a taxa no Keras

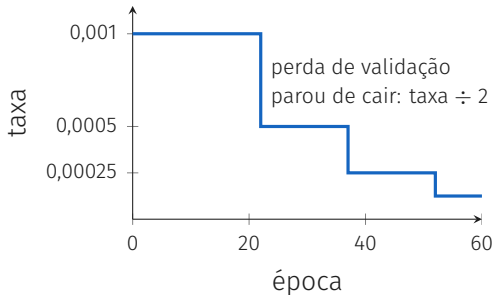
```
from keras import optimizers

modelo.compile(optimizer=optimizers.Adam(learning_rate=0.001),
               loss="binary_crossentropy",
               metrics=["accuracy"])
```

- `optimizer="adam"` usa a taxa padrão, 0,001;
- Para testar outras: valores **10 vezes** maiores ou menores.



Diminuir a taxa durante o treino



- Estacionar o carro: rápido na rua, devagar perto da vaga;
- Passos grandes no início, para chegar perto; pequenos no fim, para não passar do ponto.

```
reduzir = keras.callbacks.ReduceLROnPlateau(monitor="val_loss",  
                                             factor=0.5,      # taxa vezes 0,5  
                                             patience=3)    # espera 3 epocas  
modelo.fit(X_train, y_train, epochs=100, validation_split=0.2,  
          callbacks=[reduzir])
```

SGD (descida do gradiente com lotes)

- O mesmo tamanho de passo para todos os pesos;
- Mais sensível à escolha da taxa;
- Com **momentum**: ganha “inércia”, como uma bola descendo, e zigue-zagueia menos.

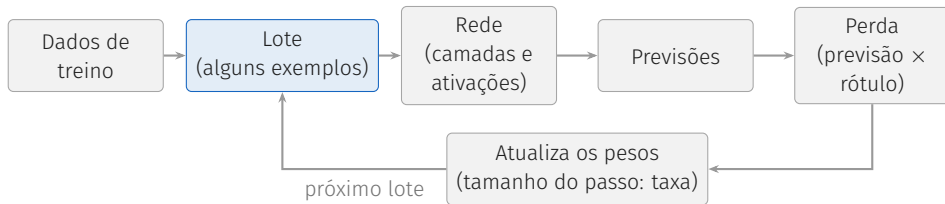
Adam

- Ajusta o tamanho do passo de **cada peso** conforme o histórico daquele peso;
- Tem inércia, como o momentum;
- Funciona bem com a taxa padrão na maioria dos casos;
- **Na prática: comecem com Adam e a taxa padrão.**

Otimizador	<i>optimizer=</i>	Taxa padrão
SGD	<i>"sgd"</i>	0,01
RMSprop	<i>"rmsprop"</i>	0,001
Adam	<i>"adam"</i>	0,001

Lote e época

Lote: uma pesquisa de opinião



todos os lotes uma vez = **1 época**; o treino repete por várias épocas

- Antes de cada passo, a rede “pergunta” aos exemplos para que lado mexer os pesos;
- Perguntar a **todos**: direção exata, mas um passo só por época. A uma **amostra** de 32: direção aproximada, muito mais passos.

Para discutir

Numa pesquisa eleitoral, por que não entrevistar uma pessoa só? E por que não o país inteiro?

Para discutir

Numa pesquisa eleitoral, por que não entrevistar uma pessoa só? E por que não o país inteiro?

- **Uma pessoa só:** o resultado depende demais de quem foi sorteado. É o lote de 1: cada passo vai para um lado diferente;
- **O país inteiro:** resultado exato, mas caro e demorado. É usar todos os exemplos: um passo só por época;
- **Uma amostra** de algumas centenas já dá uma boa ideia, e rápido. É o lote de 32: direção aproximada, muitos passos por época.

Lote e época, desenhados

dados de treino



- 12 exemplos, *batch_size*=4: 3 lotes, 3 atualizações dos pesos por época;

Lote e época, desenhados



- 12 exemplos, *batch_size*=4: 3 lotes, 3 atualizações dos pesos por época;
- Época: cada exemplo é usado uma vez;

Lote e época, desenhados



- 12 exemplos, *batch_size*=4: 3 lotes, 3 atualizações dos pesos por época;
- **Época**: cada exemplo é usado uma vez;
- A cada época, o Keras **embaralha** os exemplos: os lotes mudam. Em 2 épocas, 6 atualizações.

Contando atualizações

atualizações por época = exemplos $\div$ tamanho do lote, **arredondando para cima**

Exemplos	Lote	Atualizações por época	Em 20 épocas
1.000	100		
10	4		
1.000	32		
60.000	32		

Contando atualizações

atualizações por época = exemplos $\div$ tamanho do lote, **arredondando para cima**

Exemplos	Lote	Atualizações por época	Em 20 épocas
1.000	100	10	200
10	4		
1.000	32		
60.000	32		

Contando atualizações

atualizações por época = exemplos $\div$ tamanho do lote, **arredondando para cima**

Exemplos	Lote	Atualizações por época	Em 20 épocas
1.000	100	10	200
10	4	3 (lotes de 4, 4 e 2)	60
1.000	32		
60.000	32		

Contando atualizações

atualizações por época = exemplos $\div$ tamanho do lote, **arredondando para cima**

Exemplos	Lote	Atualizações por época	Em 20 épocas
1.000	100	10	200
10	4	3 (lotes de 4, 4 e 2)	60
1.000	32	32 (31 lotes de 32 e 1 de 8)	640
60.000	32		

Contando atualizações

atualizações por época = exemplos $\div$ tamanho do lote, **arredondando para cima**

Exemplos	Lote	Atualizações por época	Em 20 épocas
1.000	100	10	200
10	4	3 (lotes de 4, 4 e 2)	60
1.000	32	32 (31 lotes de 32 e 1 de 8)	640
60.000	32	1.875	37.500

60.000 é o tamanho do treino do MNIST, os dígitos escritos à mão. Os nomes **passo** e **iteração** também são usados para uma atualização.

Lendo a saída do `fit`

```
modelo.fit(X_train, y_train, epochs=10, batch_size=32,  
           validation_split=0.2)
```

```
Epoch 1/10
```

```
1500/1500 - loss: 0.4410 - accuracy: 0.8721
```

```
- val_loss: 0.2068 - val_accuracy: 0.9405
```

```
Epoch 2/10
```

```
1500/1500 - loss: 0.1806 - accuracy: 0.9476
```

```
- val_loss: 0.1481 - val_accuracy: 0.9572
```

- *Epoch 1/10*: a primeira de 10 épocas;
- *1500/1500*: lotes da época, ou seja, atualizações dos pesos;
- *loss, accuracy*: média no treino, durante a época;
- *val_loss, val_accuracy*: na validação, no fim da época.

Palpite: 1500 ou 1875?

Palpite, antes de ver o resultado

O MNIST tem 60.000 imagens de treino e o lote é 32: $60.000 \div 32 = 1.875$. Mas a saída do *fit* mostra **1500/1500**. Por quê?

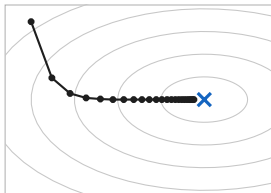
Palpite, antes de ver o resultado

O MNIST tem 60.000 imagens de treino e o lote é 32: $60.000 \div 32 = 1.875$. Mas a saída do *fit* mostra **1500/1500**. Por quê?

- *validation_split=0.2* separa 20% das imagens para a validação, antes de treinar;
- Sobram 48.000 imagens para o treino: $48.000 \div 32 = 1.500$ lotes por época;
- A validação não atualiza os pesos: ela só é avaliada no fim de cada época.

O tamanho do lote muda o caminho

todos os exemplos



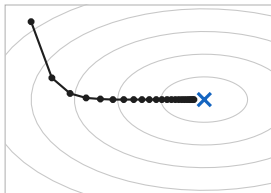
direção exata

1 passo por época

- Vista de cima: as elipses são curvas de nível da perda, com dois pesos; o $\times$ é o mínimo. Cada ponto é um passo;

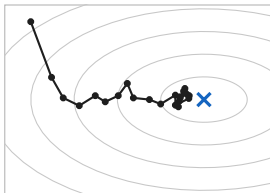
O tamanho do lote muda o caminho

todos os exemplos



direção exata
1 passo por época

lote de 32

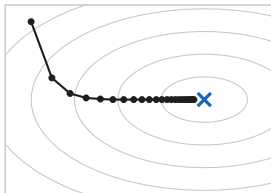


um pouco de ruído
muitos passos por época

- Vista de cima: as elipses são curvas de nível da perda, com dois pesos; o $\times$ é o mínimo. Cada ponto é um passo;

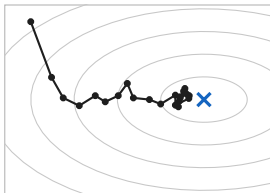
O tamanho do lote muda o caminho

todos os exemplos



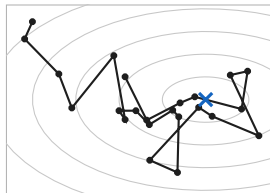
direção exata
1 passo por época

lote de 32



um pouco de ruído
muitos passos por época

lote de 1



muito ruído
um passo por exemplo

- Vista de cima: as elipses são curvas de nível da perda, com dois pesos; o $\times$ é o mínimo. Cada ponto é um passo;
- Lote pequeno: direção ruidosa. Um pouco de ruído não atrapalha, e às vezes ajuda a escapar de lugares ruins.

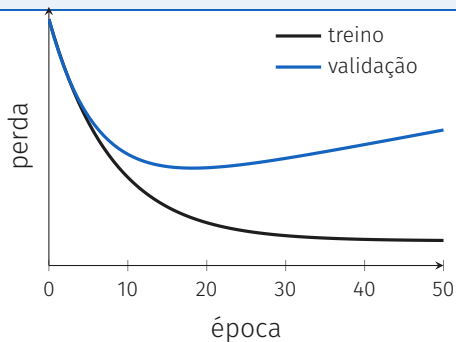
Escolhendo o tamanho do lote

48.000 exemplos de treino:

<i>batch_size</i>	Atualizações por época	Ruído na direção	Memória
1	48.000	muito	pouca
32	1.500	moderado	moderada
512	94	pouco	muita
48.000	1	nenhum	toda a base de uma vez

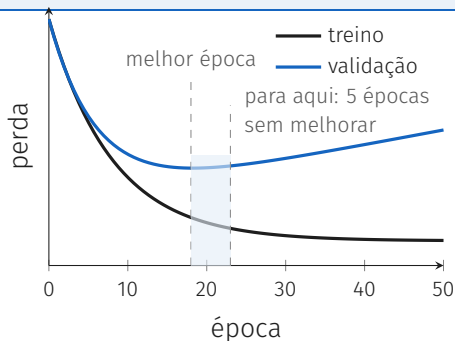
- Comecem com **32**, o padrão do Keras;
- Lote maior: cada época roda mais rápido numa GPU, mas a rede dá menos passos por época;
- Ao aumentar muito o lote, costuma ser preciso aumentar a taxa também;
- “Erro de memória” (*out of memory*) na GPU: diminuam o lote.

Quantas épocas?



- Poucas épocas: a rede ainda está aprendendo;
- Muitas: **sobreajuste**, a validação piora (Aula 2);

Quantas épocas?



- Poucas épocas: a rede ainda está aprendendo;
- Muitas: **sobreajuste**, a validação piora (Aula 2);
- **Parada antecipada**: pede muitas épocas e para quando a validação não melhora há *patience* épocas.

```
parar = keras.callbacks.EarlyStopping(  
    monitor="val_loss", patience=5,  
    restore_best_weights=True)          # volta a melhor epoca  
modelo.fit(X_train, y_train, epochs=200,  
           validation_split=0.2, callbacks=[parar])
```

Pergunta: contando atualizações

Uma rede treina com 2.000 exemplos, *batch_size=50* e *epochs=10*. Quantas vezes os pesos são atualizados?

- a) 10;
- b) 40;
- c) 400;
- d) 20.000.

Pergunta: contando atualizações

Uma rede treina com 2.000 exemplos, *batch_size=50* e *epochs=10*. Quantas vezes os pesos são atualizados?

- a) 10;
- b) 40;
- c) 400;
- d) 20.000.

Resposta: c. $2.000 \div 50 = 40$ lotes por época, vezes 10 épocas. 40 (b) é uma época só. 20.000 (d) seria com lote de 1: uma atualização por exemplo.

Pergunta: lote maior

Um colega aumentou o *batch_size* de 32 para 256. Cada época ficou mais rápida, mas a perda caiu **menos por época**. Por quê?

- a) Lotes grandes causam sobreajuste;
- b) A rede dá 8 vezes menos passos por época;
- c) A taxa de aprendizado diminuiu automaticamente;
- d) Com lote grande, a rede vê menos exemplos por época.

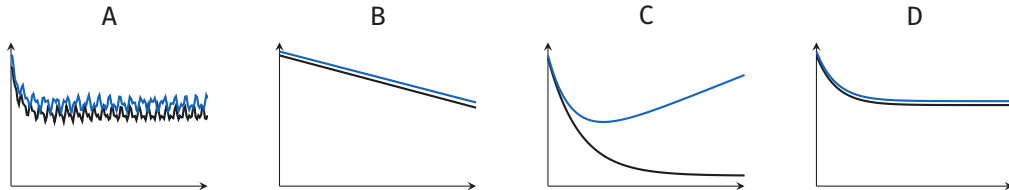
Pergunta: lote maior

Um colega aumentou o *batch_size* de 32 para 256. Cada época ficou mais rápida, mas a perda caiu **menos por época**. Por quê?

- a) Lotes grandes causam sobreajuste;
- b) A rede dá 8 vezes menos passos por época;
- c) A taxa de aprendizado diminuiu automaticamente;
- d) Com lote grande, a rede vê menos exemplos por época.

Resposta: b. Cada época continua usando todos os exemplos (d está errada), mas em 8 vezes menos lotes: 8 vezes menos atualizações. Opções: mais épocas, ou uma taxa maior. O Keras não muda a taxa sozinho (c).

Atividade: leia a curva



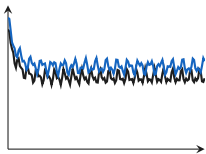
— treino — validação.

Para discutir

O que está acontecendo em cada uma? O que vocês mudariam?

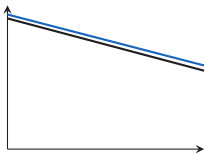
Atividade: leia a curva

A

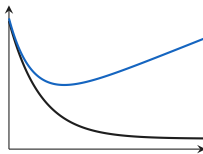


taxa alta: perda pulando num valor alto
→ taxa 10× menor

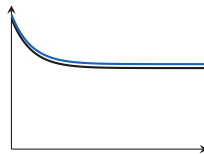
B



C



D



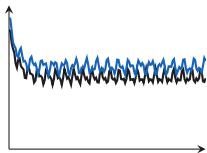
— treino — validação.

Para discutir

O que está acontecendo em cada uma? O que vocês mudariam?

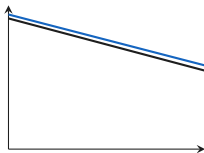
Atividade: leia a curva

A



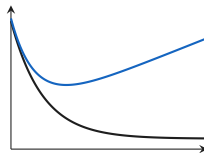
taxa alta: perda pulando num valor alto
→ taxa 10× menor

B

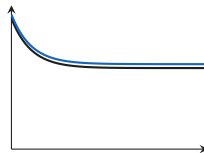


taxa baixa: desce devagar, sem parar
→ taxa 10× maior, ou mais épocas

C



D



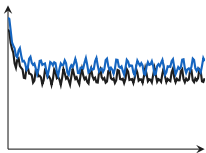
— treino — validação.

Para discutir

O que está acontecendo em cada uma? O que vocês mudariam?

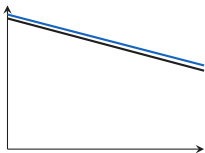
Atividade: leia a curva

A



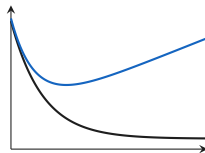
taxa alta: perda pulando num valor alto
→ taxa 10× menor

B



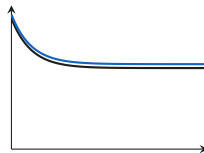
taxa baixa: desce devagar, sem parar
→ taxa 10× maior, ou mais épocas

C



sobreajuste: validação sobe
→ parada antecipada, rede menor, mais dados

D



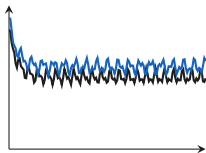
— treino — validação.

Para discutir

O que está acontecendo em cada uma? O que vocês mudariam?

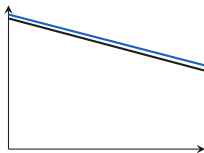
Atividade: leia a curva

A



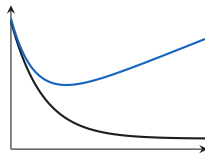
taxa alta: perda pulando num valor alto
→ taxa 10× menor

B



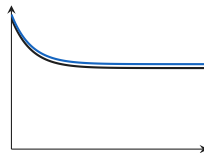
taxa baixa: desce devagar, sem parar
→ taxa 10× maior, ou mais épocas

C



sobreajuste: validação sobe
→ parada antecipada, rede menor, mais dados

D



viés alto: treino e validação pararam altos e juntos
→ rede maior, variáveis melhores

— treino — validação.

Para discutir

O que está acontecendo em cada uma? O que vocês mudariam?

Juntando tudo

Ponto de partida

Escolha	Comecem com	Mudem quando
Ativação oculta	<i>"relu"</i>	quase nunca
Ativação da saída e perda	a tabela da tarefa	a tarefa mudar
Otimizador e taxa	<i>Adam</i> , 0,001	perda explode ou pula: 10× menor. Cai devagar: 10× maior
Lote	32	falta memória: menor. Treino lento e sobra memória: maior
Épocas	muitas, com <i>EarlyStopping</i>	—

Mudem **uma coisa de cada vez** e comparem pela [validação](#).

O código, com todos os botões

```
modelo = keras.Sequential([
    keras.Input(shape=(8,)),
    layers.Dense(32, activation="relu"),          # (1) ativacao oculta
    layers.Dense(16, activation="relu"),
    layers.Dense(3, activation="softmax"),        # (2) ativacao da saida
])
modelo.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001), # (3) taxa
    loss="sparse_categorical_crossentropy",             # (4) perda
    metrics=["accuracy"])
parar = keras.callbacks.EarlyStopping(monitor="val_loss", patience=5,
                                       restore_best_weights=True)
historico = modelo.fit(X_train, y_train,
    batch_size=32,                                # (5) lote
    epochs=200, callbacks=[parar],                 # (6) epocas
    validation_split=0.2)
```

Na próxima aula, este código no Keras, com dados reais.

Estudos de caso

Em grupos, 10 min. Para cada caso, preencham: tipo de tarefa, última camada, perda e uma métrica.

- A. Quantas chamadas o 190 vai receber na próxima hora, por região;
- B. A chamada é trote? (5% das chamadas são trote);
- C. A natureza da ocorrência, entre 8 tipos, a partir do texto do relato;
- D. Em cada imagem de câmera: há pessoa? veículo? moto?

Caso	Tarefa	Última camada	Perda	Métrica
A				
B				
C				
D				

Caso A: chamadas na próxima hora

- Regressão: *Dense(1)*, sem ativação;
- Perda "*mse*"; métrica "*mae*" ("erra 4 chamadas em média").

Para discutir

1. A rede pode prever -3 chamadas? Isso é um problema?
2. No réveillon há picos enormes. MSE ou MAE?

Caso A: chamadas na próxima hora

- Regressão: *Dense(1)*, sem ativação;
- Perda "*mse*"; métrica "*mae*" ("erra 4 chamadas em média").

Para discutir

1. A rede pode prever -3 chamadas? Isso é um problema?
 2. No réveillon há picos enormes. MSE ou MAE?
- Sem ativação, a saída pode ser negativa. Se acontecer, dá para cortar em zero na hora de usar, ou usar ReLU na saída;
 - MSE faz os picos pesarem muito: bom se errar no réveillon for grave; ruim se forem casos raros que distorcem o resto.

Caso B: a chamada é trote?

- Classificação binária: `Dense(1, activation="sigmoid");`
- Perda `"binary_crossentropy"`.

Para discutir

Um modelo que responde sempre “não é trote” tem que acurácia? O que isso diz sobre usar só a acurácia aqui?

Caso B: a chamada é trote?

- Classificação binária: `Dense(1, activation="sigmoid");`
- Perda `"binary_crossentropy"`.

Para discutir

Um modelo que responde sempre “não é trote” tem que acurácia? O que isso diz sobre usar só a acurácia aqui?

95% de acurácia, sem aprender nada. Com classes desbalanceadas, olhem também precisão e recall da classe rara. Na Aula 5 voltamos a isso.

Caso C: natureza da ocorrência

- Multiclasse: `Dense(8, activation="softmax");`
- Perda `"sparse_categorical_crossentropy"`, com rótulos de 0 a 7;
- Métrica `"accuracy"`.

Para discutir

1. Os rótulos estão como texto: "furto", "roubo"... O que fazer antes de treinar?
2. Uma ocorrência pode ter duas naturezas? Se puder, o que muda?

Caso C: natureza da ocorrência

- Multiclasse: `Dense(8, activation="softmax");`
- Perda `"sparse_categorical_crossentropy"`, com rótulos de 0 a 7;
- Métrica `"accuracy"`.

Para discutir

1. Os rótulos estão como texto: "furto", "roubo"... O que fazer antes de treinar?
2. Uma ocorrência pode ter duas naturezas? Se puder, o que muda?

- Converter o texto em inteiros de 0 a 7, ou em one-hot (e aí `"categorical_crossentropy"`);
- Se puder ter duas: vira multirrótulo, como o caso D.

Caso D: o que há na imagem

- Multirrótulo: `Dense(3, activation="sigmoid")`, uma sigmoide por etiqueta;
- Perda `"binary_crossentropy"`: um “sim ou não” para cada etiqueta.

Para discutir

Por que não softmax? O que a rede diria numa imagem com uma pessoa ao lado de um carro?

Caso D: o que há na imagem

- Multirrótulo: `Dense(3, activation="sigmoid")`, uma sigmoide por etiqueta;
- Perda `"binary_crossentropy"`: um “sim ou não” para cada etiqueta.

Para discutir

Por que não softmax? O que a rede diria numa imagem com uma pessoa ao lado de um carro?

Com softmax, as três probabilidades somam 1: pessoa e veículo teriam que dividir a certeza, por exemplo 0,5 e 0,5. Com sigmóides, os dois podem ter 0,9. Imagens: Aula 4, com redes convolucionais.

Fechamento

Kahoot

Aulas 1, 2 e 3 e o conteúdo de ML.

Termos da aula

Função de ativação	função aplicada depois da soma ponderada de cada neurônio
ReLU	zera o negativo; o padrão nas camadas ocultas
Sigmoide	espreme entre 0 e 1; saída de “sim ou não”
Softmax	transforma pontuações em probabilidades que somam 1
Função de perda	número que mede o erro; o que o treino minimiza
MSE e MAE	média dos erros ao quadrado; média dos erros sem sinal
Entropia cruzada	perda de classificação: a surpresa com a resposta certa

Métrica	número para as pessoas lerem; não é usado no treino
Taxa de aprendizado	tamanho de cada passo da descida
Otimizador	a regra do passo: SGD, Adam
Lote	exemplos usados em cada atualização dos pesos
Época	uma passagem completa pelos dados de treino
Passo, iteração	uma atualização dos pesos
Parada antecipada	parar quando a validação deixa de melhorar

- Qual é o tipo de tarefa do projeto: regressão, binária, multiclasse ou multirrótulo?
- Qual será a última camada e qual será a perda?
- Os rótulos estão como texto, inteiros ou one-hot?
- Na regressão: há valores atípicos ou erros de registro na resposta? MSE ou MAE?
- Qual métrica vocês vão relatar? Ela mede o erro que importa no uso real?

O que levar desta aula

1. **Ativação nas camadas ocultas:** *"relu"*. Cada neurônio ReLU acrescenta uma dobra; sem ativação, a rede é uma reta;
2. **Ativação na saída:** escolhida pelo tipo de resposta. Número: nenhuma. Sim ou não: *"sigmoid"*. Uma classe entre K : *"softmax"*, com K neurônios. Várias etiquetas: K sigmoides;
3. **Perda:** anda junto com a saída. Regressão: *"mse"* (erros grandes pesam mais) ou *"mae"* (resiste a valores atípicos). Classificação: entropia cruzada, com *sparse_* para rótulos inteiros;
4. **Taxa de aprendizado:** o tamanho do passo. Comecem com Adam e 0,001. Se a perda explode, 10× menor; se cai devagar, 10× maior;
5. **Lote e época:** atualizações por época = exemplos $\div$ lote. Comecem com lote 32 e muitas épocas, com *EarlyStopping* olhando a validação.

- CHOLLET, F. *Deep Learning with Python*. 2. ed. Manning, 2021.
- PRINCE, S. J. D. *Understanding Deep Learning*. MIT Press, 2023. Caps. 3 (ReLU e dobras), 5 (funções de perda) e 6 (otimização).
- GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. MIT Press, 2016. Caps. 6 e 8.
- GLOROT, X.; BORDES, A.; BENGIO, Y. Deep sparse rectifier neural networks. AISTATS, 2011.
- KINGMA, D. P.; BA, J. Adam: a method for stochastic optimization. ICLR, 2015.
- KARPATY, A. et al. CS231n: Convolutional Neural Networks for Visual Recognition. Notas de aula, Stanford.